

Day 7: Power Query + M Language Mastery

📌 Definition to Remember

M (Mashup Language) is the functional, case-sensitive programming language that powers all Power Query ETL operations behind the graphical user interface.

1. Structure of an M Query: `let ... in`

Every M expression is built around a `let` block (defining step variables) and an `in` block (returning final output):

```
let
    // Step 1: Connect to Source
    Source = Csv.Document(File.Contents("C:\Data\Sales.csv"), [Delimiter=";", Columns=4,
    Encoding=65001]),

    // Step 2: Promote Headers
    PromotedHeaders = Table.PromoteHeaders(Source, [PromoteAllScalars=true]),

    // Step 3: Change Column Types
    ChangedTypes = Table.TransformColumnTypes(PromotedHeaders, {
        {"OrderID", Int64.Type},
        {"SalesAmount", type number},
        {"OrderDate", type date}
    }),

    // Step 4: Filter Rows
    FilteredRows = Table.SelectRows(ChangedTypes, each [SalesAmount] > 0)
in
    FilteredRows
```

2. Core M Language Concepts

- Case Sensitivity:** Function names and column identifiers are strictly case-sensitive (`Table.SelectRows` != `table.selectrows`).
- Immutable Variables:** Each step produces a new table variable reference; variables cannot be overwritten.
- Essential M Functions:**
 - `Table.SelectRows()` , `Table.TransformColumnTypes()` , `Table.NestedJoin()`
 - `Text.Proper()` , `Text.Upper()` , `Text.BetweenDelimiters()`
 - `Date.Year()` , `Date.Month()` , `Date.EndOfMonth()`

3. Writing Custom Reusable M Functions

Converting a sequence of cleaning steps into a parameterized reusable function:

```
(inputDate as date) as text =>
let
    FiscalYear = if Date.Month(inputDate) >= 4
                  then "FY" & Text.End(Text.From(Date.Year(inputDate) + 1), 2)
                  else "FY" & Text.End(Text.From(Date.Year(inputDate)), 2)
in
    FiscalYear
```

★ Must-Master Skills & Interview Points

1. Understand the **let ... in** syntax hierarchy.
2. How to use Advanced Editor to debug broken ETL steps and parameterize file paths.
3. Creating custom M functions for automated date and text parsing.

⚡ Quick Recall

let (step definitions) in (output result) – Case-sensitive, functional, non-destructive